AI Trust Framework Requirements

Regulation asks its question **at event time**. A nightly batch cannot say what someone consented to at 14:32. **Ten requirements**, where REST falls short structurally, and **nine additive changes** that need no endpoint rewritten.

For: **teams with a working REST integration layer who now have to make it defensible.**

Premise: **you are not going to rewrite it. Everything below is additive.**

Revised: **23 October 2026 — calendar updated for the AI Omnibus, the TLS lifetime schedule and the eMudhra distrust; rotation recommendations added.**

Companions: The Trust Framework for the non-technical case; REST Is Not GraphQL for the migration mechanics; Key Custody for how an agent holds a key; the Key Management Lifecycle for Dev, Stage and Prod; PHP Commerce: Moving Off Shared Hosting for the October 2026 breaches, signed records and the key ceremony.

The gap, in one sentence

Shopify and Google moved the trust boundary server-side — consent and attribution are now **per-event facts resolved at the moment of effect**. Most PIM, CRM and ERP integration stayed on nightly batches, flat files and REST syncs, a shape whose founding assumption is that **state can be reconciled later**.

Regulation asks its question at event time. A nightly batch cannot say what someone consented to at 14:32, because at 14:32 it was not there.

That is the whole problem. Everything below follows from it.

Why this is now expensive

Not one deadline — a wave, which is worse, because there is no single date to plan around:

INSTRUMENT	IN FORCE	HARD OBLIGATIONS	STATUS AT 23 OCT 2026	EXPOSURE
GDPR	2018	live	live	4% global turnover
NIS2	transposition 17 Oct 2024	live in member states	live	personal management liability
EU AI Act	1 Aug 2024	prohibitions Feb 2025 · GPAI Aug 2025 · high-risk Annex III **2 Dec 2027** · Annex I **2 Aug 2028**	high-risk dates moved by the AI Omnibus, Regulation (EU) 2026/1744, in force 27 Jul 2026; prohibitions and GPAI duties unchanged	€35M or 7% global turnover
Data Act	Jan 2024	applies 12 Sept 2025	live	member-state penalties
Cyber Resilience Act	10 Dec 2024	reporting 11 Sept 2026 · full 11 Dec 2027	**reporting live**: 24-hour warning, 72-hour notification, 14-day final report	€15M or 2.5%
DSA	Feb 2024	live	live	6% global turnover

The certificate calendar runs beside it. These are not regulations, but they set how often keys and certificates must change, and they arrive on fixed dates:

DATE	WHAT CHANGES	WHAT IT ASKS OF YOU
15 Mar 2026	Publicly trusted TLS certificates capped at 200 days (CA/Browser Forum ballot SC-081)	Automated issuance; a manual renewal now comes round twice a year
17–27 Oct 2026	eMudhra/emSign intermediates distrusted after weak keys were found; Chrome removes the root 27 Oct	Patched runtimes and trust stores (Node, Go, Python's certifi and pip); reissue anything chained to emSign
15 Mar 2027	TLS cap falls to 100 days	Renewal must need no person
2 Dec 2027	AI Act high-risk (Annex III) obligations apply	Mandates, provenance and human accountability (R3, R9, R10) in place
11 Dec 2027	Cyber Resilience Act fully applies	Signed releases, SBOMs and a secure update path you can attest
2 Aug 2028	AI Act high-risk (Annex I, embedded in regulated products)	As above, for AI inside products
15 Mar 2029	TLS cap falls to 47 days; domain validation reuse to 10 days	ACME end to end

The AI Omnibus moved dates; it did not remove obligations. The work is the same, with more time to do it properly — and the Cyber Resilience Act reporting clock is already running.

The AI Act matters most here for a reason that is easy to miss: its obligations attach to **deployers**, not only to model builders. Using a general-purpose model inside a high-risk workflow makes the obligations yours. Personal liability under NIS2 is the detail that changes who in your organisation cares.

What actually changed, and it is not the existence of penalties

Large fines predate this wave — Meta took €1.2B in 2023, Amazon €746M in 2021. The shift is in **what a regulator now asks for**.

The earlier regime was principles-based: you interpreted a standard, documented your reasoning, and argued it. A defensible interpretation was a defensible position. The current instruments are **specific, dated and evidentiary** — named obligations, fixed deadlines, prescribed records, and conformity assessments you either produce or do not.

Three consequences follow, and they are what make this expensive rather than merely serious:

Argument stops working. You cannot interpret your way out of an obligation to hold a record. Either the record exists, with the right fields, from the right date, or it does not. Reasonable intent is no longer responsive to the question.

Ceilings stack rather than substitute. One course of conduct can engage GDPR, the AI Act and NIS2 simultaneously, each with its own ceiling and its own supervisory authority. Organisations model exposure against a single instrument and get the arithmetic wrong.

Enforcement follows evidence, and technology firms generate the most of it. Not because regulators target them by sector, but because the obligations concentrate where data volume, automated decisions and cross-border transfers concentrate — and that is software. The enforcement record reflects it: the largest actions to date have landed on platform and technology businesses, and the newer instruments were drafted with exactly those operating models in view.

If your organisation builds or sells software, the practical reading is straightforward. You are in the population these instruments were written about, the obligations are evidentiary rather than interpretive, and the evidence has to exist before the question arrives.

Part 1 — The ten requirements

A framework is conformant if it can satisfy all ten. Most stacks satisfy four.

R1 · Event-time resolution

Consent, entitlement and authority are evaluated **at the moment of effect**, not inherited from a session and not reconciled overnight. If the answer can change between the check and the effect, the check happened in the wrong place.

R2 · Per-actor identity

Every actor — person, service, and agent — carries a distinct identity. A shared service credential makes attribution unrecoverable, because the information was never captured. This is the requirement that cannot be retrofitted from logs.

R3 · Bounded, revocable authority

Authority is granted with scope, a cap, and an expiry, and can be revoked individually without disturbing anything else. An agent holds a mandate, never a principal's credential.

R4 · Append-only record

Entries are added, never edited. A correction is a new entry referencing the old one. If any entry can be silently amended, none of them prove anything — including the correct ones.

R5 · Temporal queryability

The system answers "**what was true at time T**", not only "what is true now". Almost every regulatory question is retrospective: the price 30 days before

the promotion, who held access in March, what the policy said when the incident occurred.

R6 · Independent verifiability

A third party can confirm a record without trusting you and without calling an API you control. Publish a key; sign the records; let them check offline.

R7 · Jurisdictional shape

The same entity carries different obligations per market — consent basis, retention, disclosure, price-history duties. Jurisdiction is a first-class field on the record, not something inferred from a storefront domain at read time.

R8 · Denials are recorded

A refused action is evidence, often the best evidence you have. A framework that logs only what happened cannot demonstrate that controls were operating.

R9 · Provenance of automated change

For anything an agent did: which agent, under which mandate, against which policy or prompt version, with which inputs. "The system updated it" is not an answer anyone accepts.

R10 · Bound human accountability

Every mandate names an accountable human. This is not ceremony — it is the operative requirement behind AI Act human-oversight duties, and it is what converts an autonomous action into an attributable one.

Part 2 — Where REST falls short structurally

Not defects. Design choices that were correct for their era and are now load-bearing in the wrong direction.

Resources return current state. `GET /customers/42` answers *now*. There is no standard shape for *as of last March* — so R5 has nowhere to live.

A bearer token is authentication, not authority. The request carries "who", not "who, for what scope, up to what cap, until when." R3 has no slot in the envelope.

Polling and batch encode the reconciliation assumption. Any cadence longer than zero says state may be temporarily wrong and will be fixed later. That is incompatible with R1 by construction, not by configuration.

No ordering guarantee. Without sequence or idempotency, a replayed or reordered batch is undetectable — and R4's chain cannot be established after the fact.

Partial success is invisible. A batch of 5,000 rows where 12 failed typically surfaces as one status code. R8 and R9 both die here.

Errors describe transport, not decision. `403` records that something was refused, not what rule refused it, or under whose authority the attempt was made.

The honest summary: REST can carry all of this. It just does not carry any of it **by default**, and defaults are what shipped.

Part 3 — Nine additive changes

None of these require touching an existing endpoint's contract. Ordered by ratio of obligation closed to effort spent.

1 · Put a ledger beside your resources

Add one append-only table. Every consequential write emits a row: actor, action, subject, scope, jurisdiction, decision, timestamp, and the hash of the previous row. Your endpoints keep their shapes; the ledger is written alongside.

This one change alone provides R4, most of R5, and the substrate for R8 and R9.

```
ledger(id, prev_hash, hash, ts, actor_id, actor_type, mandate_id,
       action, subject_ref, scope, jurisdiction, decision,
       consent_event_id, policy_version, payload_digest)
```

Hash each row over its own fields plus `prev_hash`. Chain integrity then makes silent amendment detectable, which is what turns a log into evidence.

2 · Carry actor identity on every write

Add `actor_id` and `actor_type` (`human` | `service` | `agent`) to your write paths. Stop letting anything write as the integration itself. **R2** — and note this is the one item on the list that gets strictly harder every day you defer it, because unattributed history stays unattributed forever.

3 · Store consent by reference, never as a boolean

Replace `marketing_consent: true` with `consent_event_id`, pointing at an immutable row carrying timestamp, mechanism, policy version, purpose and jurisdiction. The boolean becomes a derived read.

Satisfies **R7** and the part of **R5** that regulators ask about most.

4 · Add one as-of read path

A single endpoint — `GET /…/as-of?t=<iso>` — that reconstructs state from the ledger. You do not need it on every resource. You need it on the ones you will be asked about: prices, consent, entitlement, access.

R5, at the cost of one handler.

5 · Resolve authority in middleware, before the effect

A pre-write check that resolves scope, cap and expiry for the acting identity, and **writes a ledger row whether it permits or refuses**. Refusals are the evidence that controls were live.

R1, R3, R8 in one insertion point — which is why this is the highest-leverage change on the list after the ledger itself.

6 · Idempotency keys and a monotonic sequence

Every write carries a client-generated idempotency key; every ledger row carries a sequence number. Replays become detectable, ordering becomes reconstructible, and batch reconciliation stops being archaeology.

7 · Sign consequential records, publish the key

Ed25519 over the ledger row; publish a JWKS endpoint. A counterparty, auditor or customer verifies **offline**, without an account and without trusting your answer.

R6. Roughly a day of work, and it is the requirement that most changes the character of a dispute — because you stop being the sole custodian of your own evidence.

8 · Make jurisdiction first-class

A market field on the record, set at write time from the transaction context — not derived at read time from a domain, a currency, or an IP address. Derivation at read time is exactly how the same order acquires two different regulatory shapes in two different reports.

R7.

9 · Bind an accountable human to every mandate

The mandate record carries a named person. When an agent acts, the ledger row carries the mandate, and therefore carries the human. **R10** — and this is the field that makes an autonomous action defensible rather than merely explicable.

The erasure problem: append-only records and GDPR erasure

Append-only records and a GDPR erasure request appear to be in direct conflict. They are not, and the resolution is standard:

Keep personal data out of the ledger. Ledger rows carry *references* and *digests*, never personal fields. The personal data lives in an erasable store keyed by that reference. Erasure removes the referenced record and writes a tombstone; the chain stays intact, the hashes still verify, and what remains is the fact that an event occurred, its time, and its authority — which is exactly what a regulator wants and contains nothing they would object to retaining.

Design this on day one. Retrofitting it means rebuilding the chain.

Firmware identity: the part-number gap

Requirements R5, R6 and R9 assume you can say what a record is *about*. For firmware, that is harder than it looks. A buyer orders by manufacturer part number (MPN). Firmware is tied to the device by technical identifiers instead, and **there is no standard link between the two.** The mapping usually lives in the manufacturer's internal systems.

Where each identifier lives today:

WHERE	WHAT IDENTIFIES THE DEVICE	DOES THE MPN APPEAR?
The firmware update itself	UEFI capsules carry a firmware class GUID, one per model or component. IETF SUIT manifests (IoT) carry a vendor ID and a class ID, ideally a UUID derived from vendor plus model. Windows Update and LVFS/fwupd match hardware IDs computed from SMBIOS fields (manufacturer, product name, family, SKU). Components match on bus IDs: PCI vendor/device/subsystem, USB VID/PID	No. The update says "I'm for class GUID X"; the device says "I'm class GUID X"
The device	SMBIOS tables, populated by the firmware: manufacturer, product name, SKU number	Nearly. SKU is the closest on-device equivalent, but it is free text and filled inconsistently
The platform certificate	The TCG Platform Certificate records the platform's manufacturer, model, version and serial, and each component's manufacturer, model, serial and revision	Model, not orderable MPN. It is the closest signed record of model identity to hardware configuration; firmware version appears only indirectly, as a component revision
Attestation reference values	In the IETF RATS model, a CoRIM/CoMID manifest — a signed list of known-good measurements — names a device class by vendor, model and boot layer and lists its expected firmware measurements. TCG Reference Integrity Manifests do the same for PCs. A verifier compares DICE or TPM measurements against them	No. This is where firmware hashes formally meet model identity, but "model" is the manufacturer's own string
SBOMs	CycloneDX and SPDX describe firmware by supplier, name, version, CPE or PURL; CycloneDX has device and firmware component types	Optional, and often skipped
Manufacturing and ERP	In the product's bill of materials, firmware is its own line item with its own part number and revision. A pre-programmed chip often gets a different part number from the blank one ("MCU MPN + firmware v2.3"); distributors that program chips do the same	**Yes — accurately, but privately**

The gap. The authoritative *MPN ↔ firmware version ↔ firmware hash* mapping sits inside the manufacturer's ERP and PLM systems. The public, cryptographic artifacts — capsules, SUIT manifests, CoRIM, platform certificates — speak in GUIDs and model strings that do not map cleanly to orderable part numbers.

A distributor, an integrator or a buyer validating devices has no signed way to answer: *this MPN, at this revision, should be running exactly these firmware hashes.*

That is a catalog and identity-resolution problem as much as a security one, and it is where the changes above apply directly:

- **Change 1 (ledger)** and **change 7 (sign, publish the key):** a signed registry row that binds MPN, revision, firmware version, full SHA-256 and the linked SBOM, verifiable offline. CRM Sync's firmware vault already signs product, version, digest and SBOM in each upload certificate; carrying the MPN and revision on that row is what closes the gap for a buyer.
- **Change 4 (as-of read):** "which firmware should this MPN have been running on 1 March?" is a temporal question (R5).
- **Change 2 (actor identity):** who certified the mapping — the manufacturer, a programming distributor, an integrator — is R9 provenance.

A trusted third-party issuer, eMudhra-style, could plausibly publish such mappings. The eMudhra incident is also the caution: the issuer's own keys must come from a sound random source, and the mapping must stay verifiable when the issuer is distrusted — which is why records should be signed with keys you can retire and republish, not only chained to a public CA.

See it working: the Protected Firmware Asset Management and Download Service vaults each image under its own key, certifies every upload, grants every download, and records each event in a hash-chained ledger — with SBOMs linked per image and certificates that verify against a public JWKS.

Rotation recommendations

Requirement R6 depends on keys, and keys age. Rotation limits how long a leaked key is useful; it must never break the record. Two rules hold for every key below:

- **Retire, never delete, a signing key.** A retired key's public half stays in the JWKS, marked retired, so every record it signed keeps verifying. Rotation never re-signs history.
- **The rotation is itself a ledger row:** who rotated, which key id, which environment, the fingerprint, and why. That is R4 and R10 applied to your own keys.

KEY	ROUTINE CADENCE	ROTATE NOW WHEN	NOTES
Record and mandate signing key (Ed25519, change 7)	90 days, with a published overlap	a signing host or CI runner is exposed; staff or agency with access leaves	Old `kid` stays published as retired
Session signing secret	90 days	the same, plus any log or transcript exposure	Never also the encryption key — one secret, one job
Data-encryption master key (wraps per-record keys)	yearly	any suspected exposure	Re-wrap the record keys; the data is not re-encrypted
Agent mandates	not rotated — they **expire**	the agent misbehaves or its operator changes	Hours to days, not months; revoke one without touching others (R3)
Tenant and API tokens	90 days	handoff, offboarding, a token seen anywhere outside its store	Scoped per tenant, never an admin key
Third-party platform tokens (commerce, database, payment)	as the vendor allows	the token appears in a log, CI output or AI tool response	The LiteLLM incident in Oct 2026 began with OAuth tokens in gateway logs
Webhook signing secrets	yearly	the receiving endpoint is exposed	Accept old and new during the overlap
TLS certificates	automated, within the CA/Browser Forum cap (200 → 100 → 47 days)	the issuing CA is distrusted	The trust store belongs in the SBOM

How a rotation is run.

1. **One ceremony per environment.** Dev, Stage and Prod each get their own keys; no value is copied between them. Matching fingerprints across environments mean something was copied.

2. **A person creates the key; automation never sees it.** The generator pipes straight into the secret store, and only an eight-character fingerprint is recorded.
3. **Fix trust before rotating.** After a CA or package-manager incident, patch the machine first — a key made on an unpatched machine may need rotating again.
4. **Generate from the platform's cryptographic random source**, never by hand or from a copied script. The eMudhra keys were made in an audited ceremony and were still computable.
5. **Plan for post-quantum.** NIST's draft transition (IR 8547) deprecates today's RSA and elliptic-curve signatures after 2030 and disallows them after 2035. Keep key ids and algorithm names in every record now, so an algorithm change is a new `kid`, not a migration.

The step-by-step ceremony is in the Key Management Lifecycle and PHP Commerce: the key ceremony.

Conformance checklist

Run this against your own stack. Each is answerable in an afternoon; the pattern of failures tells you where to start.

1. Can you name the individual actor behind a change made six months ago? → R2
2. Can an automation be revoked without breaking the others? → R2, R3
3. Is there anything an agent *cannot* do that is enforced rather than merely configured? → R3
4. Can any historical record be edited by an administrator without trace? → R4

5. Can you state a price, a permission or a consent state as of a specific past date? → R5
6. Could a sceptical outsider verify one of your records without your cooperation? → R6
7. Does the same transaction carry its market's obligations, or are they inferred later? → R7
8. Do you have a record of what was *refused*? → R8
9. For your last automated change: which agent, which mandate, which policy version? → R9
0. Is there a named human accountable for each automated capability? → R10
1. Could you rotate every signing key this week without a record failing to verify? → R6
2. For firmware you ship or buy: is there a signed record linking part number, revision and firmware hash? → R6, R9

Fewer than six is normal. It is also the honest starting position for a remediation plan, and a far better thing to bring to an auditor than a confident answer you cannot substantiate.

What this costs

Items 1, 2, 3 and 5 are the ones that close the most obligation, and none of them is a platform decision — they are a table, two columns, a pointer and a middleware hook. They can be built inside an existing scope, on infrastructure most organisations already run, and demonstrated against real traffic before anyone is asked to approve a budget.

That is the point worth carrying back: this is not a procurement item. It is a design decision that someone has to make, which remains the scarce resource.

The alternative is sobering

Not doing this is also a decision, and it has a shape. It is worth looking at directly, because the failure is rarely dramatic — it is a series of ordinary moments that each seem survivable.

You find out by being asked. Nobody discovers a provenance gap during a quiet quarter. It surfaces as a regulator's question, a customer's subject-access request, a partner's audit, or an incident review — and the discovery and the deadline arrive together.

What you produce is a reconstruction, and it reads like one. Assembled from exports, adjacent logs and recollection. It may even be accurate. It will not be persuasive, because everyone in the room can see it was built afterwards by the party with an interest in the conclusion. That distinction is the difference between a finding and a fine.

The remediation lands at the worst possible time. Building the ledger under a deadline, with counsel involved and a supervisory authority waiting, costs a multiple of building it on an ordinary Tuesday — and you build it anyway. The only variable was when.

Your agents stay in the demo. This is the cost nobody forecasts. Teams pilot agent workflows successfully, then cannot put them near regulated data, real money or customer records, because there is no way to bound and attribute what they do. The pilot succeeds and the deployment never happens — and it is filed as "AI didn't work for us" when what failed was the substrate underneath it.

You cannot attest, so you cannot sell. Enterprise procurement and public tenders increasingly ask for evidence of exactly these controls. An organisation that cannot answer is not penalised — it is simply not shortlisted, silently, in rooms it never enters.

Under NIS2, some of this is personal. Management liability is not corporate liability. That changes who in the building should be reading this, and it is the reason a governance conversation now finds an audience it could not find in 2022.

And the part that is least discussed: when a system with no record fails, blame does not settle on whoever designed it. It settles on whoever was standing closest — usually the person who inherited it, raised the risk, and lacked the authority to fix it. Without a record, nothing distinguishes a failure you inherited from one you caused.

That last one is why this document exists. Every requirement above protects the organisation. Most of them also protect the individual who has to answer for it — and those are the same nine changes.

The work is a table, two columns, a pointer, and a middleware hook. The alternative is explaining, under deadline, why you cannot answer a question you were always going to be asked.
